

AI usage

I used Claude (via Claude Code) as a pair programmer on this build. The working loop was the same every time: I decided what the thing needed to do, the AI argued a position and drafted faster than I type, and nothing stayed in the repo until I had read it and a test had run against it.

That loop is why this document is mostly about disagreements. The AI is a useful second opinion precisely because it will confidently propose something reasonable-but-wrong, and the value is in catching which is which. Below are the five exchanges that actually changed the outcome, including the three where I went the other way.

1. Rejected: "let the LLM do the whole evaluation"

Suggested. The first architecture I was offered was the obvious one: normalise the submission, send everything to the model with a rubric in the prompt, parse a score back. Simple, and it covers every dimension in one call.

I rejected it, and this became the central design decision instead.

Two reasons, both from actually testing the behaviour. First, **it is unreliable at the checkable things**. Asked "does this design address the six stated requirements", the model missed a genuinely-absent requirement on some runs and hallucinated coverage on others, while a keyword-set match over the design text answers that exactly, instantly, and for free. Second, **it is not reproducible**. Resubmitting the same design produced scores several points apart. A platform whose entire premise is "attempt 2 scored higher than attempt 1" cannot have that much noise in its instrument.

What I built instead. A two-part pipeline where the rubric declares a per-dimension `deterministicShare`: requirement coverage is 70% rules, trade-off reasoning is 100% model. The deterministic half is the floor: dependency-free, milliseconds, reproducible, and able to show its evidence. The model handles what only a reader can judge.

The knock-on benefit is the one I did not anticipate: because the deterministic half cannot really fail, **a model outage degrades the review instead of breaking it**. That fell out of the split rather than being designed for.

2. Accepted, then had to correct: the "more than one valid design" problem

Suggested. When I asked how to give feedback when several designs are valid, the answer was to prompt the model with "there is no single correct solution; judge on internal consistency". Correct as far as it goes, and it is in the system prompt.

Where it was incomplete. It only addressed the model. My *deterministic* checks had exactly the same bias baked in, and worse, because rules are absolute: `ConceptCoverageCheck` compares against a reference concept list, so a learner who deliberately folded pricing into `Ticket` would be marked down by a rule with no room to disagree. The AI did not flag this, presumably because I had asked about the prompt.

What I changed. Two things, both in the deterministic half:

- **Wording.** No check says a concept is missing. It says nothing was *detected* that owns it, and asks whether that was deliberate: *"A common approach is a dedicated abstraction, but folding it into an existing class can be the right call, if you did that deliberately, say which class owns it and why."*
- **Score floors.** `ConceptCoverageCheck` floors at 40, so departing from the reference costs some marks and never all of them. `RequirementCoverageCheck`, where there genuinely *is* a right answer, has no floor.

And the reason I trust it now: **every finding is labelled with its source in the UI**, and requirement coverage shows the learner's own matched words. A verdict the learner disagrees with is arguable rather than authoritative.

3. A test written by AI found a real bug in my domain logic

I asked for edge-case tests around the attempt lifecycle rather than writing only the happy paths myself. One asserted something I had not thought about: after a submission is rejected for being too thin, *what state is the attempt in?*

It failed. My `submit()` was:

```
this.transitionTo('queued');    // ← state already changed
assertSubmittable(submission);  // ← then throws
```

The transition happened before validation, so a learner whose submission was rejected would have found their attempt stuck in `queued`: uneditable, with nothing queued, and their work effectively frozen. Reversed to validate first, so a rejected submission leaves the attempt exactly as it was. The test is `attempt-lifecycle.test.ts > rejects a submission too thin to be worth evaluating`, and it asserts the attempt is still `draft` afterwards.

This is the clearest value I got from AI on this project: not the code, the *questions*. I would have tested that submitting invalid input throws. I would not have tested what it leaves behind.

4. Prompt design: three rules that exist because of observed misbehaviour

The reviewer system prompt was written iteratively against real outputs, not in one pass. Three of its five rules were added to fix specific failure modes:

Behaviour without the rule	Rule added
Graded by resemblance to the reference concepts, marking down unconventional but sound designs	<i>"Judge the design on its own internal logic... never on how closely it resembles a reference solution."</i>
Produced fluent, generic advice that could have been written without reading the submission	<i>"Every point must quote or name something specific from their submission. If you cannot point at their words, do not make the point."</i>
Re-checked requirement coverage, duplicating the rules and occasionally contradicting them	<i>"Do not comment on whether the stated requirements are covered... its findings are given to you below. Do not repeat or contradict them."</i>

Two more settings came from the same testing: **temperature 0.1**, because two identical designs getting different verdicts destroys the progress history the product is built on; and passing the **deterministic findings into the prompt**, which is what turns two evaluators into one coherent review.

I also rejected the suggestion to describe scoring as "against a perfect design". It produced uniformly harsh scores, which is demotivating and inaccurate for a 45-minute exercise. The prompt now says *"against what a strong candidate would produce in a 45-minute interview... 50 is an average attempt."*

5. Accepted with a change: the intermediate representation

Suggested. For multi-format support, one evaluator per format: `StructuredEvaluator`, `CodeEvaluator`, `DiagramEvaluator`.

Why I changed it. That multiplies: n formats $\times$ m evaluators, and every new rule has to be written three times and kept consistent. I inverted it: normalise all three formats into one `DesignModel`, and let every evaluator read only that. Formats and evaluators become independent axes: a new format is one normaliser and zero evaluator changes; a new evaluator works across all formats immediately.

The suggestion was reasonable and would have worked for three formats. It just would not have survived the fourth.

How I kept myself honest

Nothing went in on the strength of looking right. Four things in this repo exist because the verification step caught something:

- **The `submit()` ordering bug.** A lifecycle test asserted what state an attempt is left in after a submission is *rejected*. It failed: the transition to `queued` ran before validation, so a learner whose submission was too thin would have found their work frozen in a state they could not edit. Fixed by validating first.
- **A regex that silently shifted every capture group.** A cleanup pass collapsed `[, , generic, arrayOf, plain]` into `[, generic, arrayOf, plain]` in `CodeNormalizer`. Nothing looked wrong; two tests went red immediately and named it.
- **Three scoring flaws, found by using my own product.** I wrote realistic designs for all four problems by hand and ran them through the platform. The verdicts were not convincing: concept coverage counted a mention as ownership, so name-dropping `calculateFee()` scored as well as modelling a `PricingStrategy`; the pitfall check returned 100 when it found nothing, quietly lifting every abstraction score; and edge-case detection used a global word list that missed designs handling the cases in their own vocabulary. All three are now fixed, and §2 below is the philosophy that came out of it.
- **Test thresholds that were calibration, not invariants.** When retuning broke three tests, two turned out to assert arbitrary numbers. I rewrote them to assert the actual invariant (requirement coverage must equal the deterministic score when the model owns none of it) and added one asserting the weak design is blamed for the *right* dimensions, because a score that is correct by accident is not feedback.

The division of labour

I pointed, reviewed and corrected; the AI drafted. It was quickest on work where the shape was already settled: the Express/Vite/Vitest scaffolding, the repetitive DTO and Zod schema mapping, the CSS design system, and the Mermaid arrow-parsing table, which is fiddly and exactly the kind of thing a model gets right faster than a person does.

None of it landed unedited. The generated requirement signal lists were the clearest case: the synonym sets were too narrow and missed naming a real learner would reach for, like `RateCard` for pricing, so I rewrote them and then added the `problem-catalogue` test suite to stop the same narrowness creeping into any problem added later.

Where the judgement had to be mine

Three calls carry this project, and no amount of drafting speed would have produced them:

1. **Splitting evaluation by dimension rather than by evaluator** (§1). The rubric's `deterministicShare` is the whole design in one number: requirement coverage is 70% rules because it is a checklist, trade-off reasoning is 100% model because only a reader can judge an argument.
2. **Treating "more than one valid design" as a scoring problem, not a prompting one** (§2). The prompt fix was the easy half; the rules had the same bias and rules are absolute, which is why concepts are graded dedicated / folded / absent with a floor.
3. **Deciding what a learner must hand in**. The structured format exists because free prose lets someone write three fluent paragraphs while never naming a responsibility or a cardinality, which are the two things being taught. The friction is the teaching.

I used the AI to attack all three after the fact, which is where §2 and §3 came from. It is a good adversary and a poor author: it will tell you a design is fine, and it will also find the hole in it if you ask it to try.